

```

        userPassword = input('Please enter the password: ')

        newBalance = deposit(userAccountNumber, userDepositAmount, userPassword)
        if newBalance is not None:
            print('Your new balance is:', newBalance)

    elif action == 'n':
        print('New Account:')
        userName = input('What is your name? ')
        userStartingAmount = input('What is the amount of your initial deposit?
')
        userStartingAmount = int(userStartingAmount)
        userPassword = input('What password would you like to use for this
account? ')

        userAccountNumber = len(accountsList)
        newAccount(userName, userStartingAmount, userPassword)
        print('Your new account number is:', userAccountNumber)

    --- snipped additional user interface ---

print('Done')

```

Listing 1-6: Bank simulation with a list of dictionaries

With this approach, all the data associated with one account can be found in a single dictionary ❶. To create a new account, we build a dictionary and append it to the list of accounts ❷. Each account is assigned a number (a simple integer), and that account number must be supplied when doing any action with the account. For example, the user supplies their account number when making a deposit, and the `getBalance()` function uses that account number as an index into the list of accounts ❸.

This cleans things up quite a bit, making the organization of the data more logical. But each of the functions in the program must still have access to the global list of accounts. As we'll see in the next section, granting functions access to all account data raises potential security risks. Ideally, each function should only be able to affect the data of a single account.